

Exercise 3.1: Apply symmetric rank-one quasi-Newton method to solve,

$$\text{minimize } f(x) = \frac{1}{2}x^T Qx - c^T x$$

with,

$$Q = \begin{pmatrix} 5 & 2 & 1 \\ 2 & 7 & 3 \\ 1 & 3 & 9 \end{pmatrix} \text{ and } c = \begin{pmatrix} -9 \\ 0 \\ -8 \end{pmatrix}$$

Initialize the method with $x_0 = (0, 0, 0)^T$ and $B_0 = I$. Use an exact line search.

Solution. Applying symmetric rank one quasi-Newton method to solve this minimization problem begins with finding a descent direction p_k by solving the following system,

$$B_k p_k = -\nabla f(x_k).$$

Recall that in Homework #7 we proved that the step size for exact line search on a quadratic optimization problem is given by,

$$\alpha_k = \frac{-p_k^T \nabla f(x_k)}{p_k^T Q p_k}.$$

We update x using α_k from exact line search $x_{k+1} = x_k + \alpha_k p_k$. Then we compute the following to update our estimated hermitian B ,

$$s_k = x_{k+1} - x_k,$$

$$y_k = \nabla f(x_{k+1}) - \nabla f(x_k),$$

$$B_{k+1} = B_k + \frac{(y_k - B_k s_k)(y_k - B_k s_k)^T}{(y_k - B_k s_k)^T s_k}.$$

Below is an implementation of the symmetric rank-one quasi-Newton method using exact line search applied to the given problem,

Code:

```
function [xk, xklist] = srlbtquad(x0,Q,c,tol,maxiters)
% This function takes an initial value x0, a quadratic system
% Q and c, a gradient tolerance, tol and maximum number of iter
% ations, maxiters. Performs symmetric rank-one quasi-Newton method
% with exact line search.
```

```
% Default number of maximum iterations.
```

```
if nargin < 5
```

```
    maxiters = 20000;
```

```
end
```

```
% Force column vectors
```

```
xk = x0(:);
```

```

if nargout > 1
    xklist = [xk];
end

% Setting up first step
n = length(xk);
B = eye(n,n);
fxk = .5*x0'*Q*x0 - c'*x0;
dfxk = Q*x0 - c;

for k = 1:maxiters
    % Tolerance Check
    if norm(dfxk) < tol
        break
    end

    %Quasi-newton step with exact line search
    pk = - B \ dfxk;
    if dfxk' * pk >= 0.0
        fprintf(['warning: non-descent direction ' ...
                ' at step %d (use steepest-descent)\n'],k)
        pk = - dfxk;
    end
    % Exact line search
    alpha = (-pk'*dfxk)/(pk'*Q*pk);
    % Saving old values for B computations.
    oldxk = xk;
    olddfxk = dfxk;
    % Update step.
    xk = xk + alpha * pk;
    % Append latest point to list.
    if nargout > 1
        xklist = [xklist xk];
    end

    % Compute new function and gradient values
    fxk = .5*xk'*Q*xk - c'*xk;
    dfxk = Q*xk - c;

    % Update B
    sk = xk - oldxk;
    yk = dfxk - olddfxk;
    v = yk - B * sk;

    denom = v' * sk;
    if denom == 0
        break
    end
end

```

```

end
if abs(denom) < 1.0e-14
    warning(strcat('divide by near-zero in symmetric rank-one update ',...
        ' at iteration %d; convergence?'),k)
end
% symmetric rank-one update
B = B + v * (v / denom)';
%disp(B)
end
end

```

Console:

```

>> Q = [5 2 1; 2 7 3; 1 3 9];
>> c = [-9 0 -8]';
>> x0 = [0 0 0]';
>> format long
>> [xk, xklist] = srlbtquad(x0,Q,c,10^(-8))
%% Wanted to see how B's converge to Q.
2.975510204081633    1.885714285714286    3.277551020408163
1.885714285714286    2.800000000000000    3.128571428571429
3.277551020408163    3.128571428571429    6.437755102040817

3.500214445152164    0.638587517176817    2.687258749203816
0.638587517176817    5.764194026008720    4.531589043176080
2.687258749203816    4.531589043176081    7.101833907145708

5.0000000000000113    2.0000000000000026    0.999999999999942
2.0000000000000026    6.999999999999956    3.000000000000032
0.999999999999942    3.0000000000000033    8.999999999999988

```

xk =

```

-2.000000000000000
 1.000000000000000
-1.000000000000000

```

xklist =

```

0 -1.160000000000000 -2.010368785061736 -2.000000000000000
0 0 0.990116682540279 1.000000000000000
0 -1.031111111111111 -0.988090803190367 -1.000000000000000

```

□

Exercise 3.3: Let f be a strictly convex quadratic function of one variable. Prove that the secant method for minimization will terminate in exactly one iteration for any initial starting points x_0 and x_1 .

Proof. Suppose f is a strictly convex quadratic function of one variable and let x_0 and x_1 be initial points. Note that f is of the form $f(x) = ax^2 + bx + c$ where $2a > 0$ and f' is of the form $f'(x) = 2ax + b$. Applying the secant method for minimization our first iteration looks like,

$$x_2 = x_1 - \frac{x_1 - x_0}{f'(x_1) - f'(x_0)} f'(x_1).$$

Checking if $f'(x_2) = 0$ we get,

$$\begin{aligned} f'(x_2) &= 2a \left(x_1 - \frac{x_1 - x_0}{f'(x_1) - f'(x_0)} f'(x_1) \right) + b \\ &= 2a \left(x_1 - \frac{x_1 - x_0}{2a(x_1) + b - (2a(x_0) + b)} (2a(x_1) + b) \right) + b \\ &= 2a \left(x_1 - \frac{x_1 - x_0}{2a(x_1) - 2a(x_0)} (2a(x_1) + b) \right) + b \\ &= 2a \left(x_1 - \frac{x_1 - x_0}{2a(x_1 - x_0)} (2a(x_1) + b) \right) + b \\ &= 2a \left(x_1 - \left(\frac{2a(x_1)}{2a} + \frac{b}{2a} \right) \right) + b \\ &= 2a \left(-\frac{b}{2a} \right) + b \\ &= 0 \end{aligned}$$

So the first order necessary conditions and the second order sufficient conditions are satisfied and we conclude that x_2 is a local minimizer. $\square$

Exercise 3.4: Let C be a symmetric matrix of rank one. Prove that C must have the form $C = \gamma ww^T$, where γ is a scalar and w is a vector of norm one.

Proof. Suppose C is a symmetric matrix of rank one. Let $C = U\Sigma V^T$ be the singular value decomposition. Since C is symmetric we know that $C = C^T$ and therefore $U\Sigma V^T = V\Sigma U^T$. It then follows that $U = U\Sigma U^T V\Sigma^{-1} = U\Sigma V^T U\Sigma^{-1} = V$, and since C is rank one Σ has one non-zero singular value, so our decomposition simplifies to $C = U\Sigma V^T = \sigma_1 uv^T$, and since $U = V$ we get $u = v$. $\square$

Exercise 3.8: Let B_{k+1} be obtained from B_k using the BFGS update formula. Prove that $B_{k+1}s_k = y_k$.

Proof. Recall that the BFGS update formula give B_{k+1} by,

$$B_{k+1} = B_k - \frac{(B_k s_k)(B_k s_k)^T}{s_k^T B_k s_k} + \frac{y_k y_k^T}{y_k^T s_k}.$$

Multiplying on the left by s_k and simplifying we get the following,

$$\begin{aligned} B_{k+1}s_k &= B_k s_k - \frac{(B_k s_k)(B_k s_k)^T s_k}{s_k^T B_k s_k} + \frac{y_k y_k^T s_k}{y_k^T s_k}, \\ &= B_k s_k - \frac{(B_k s_k)(s_k^T B_k^T s_k)}{s_k^T B_k s_k} + \frac{y_k(y_k^T s_k)}{y_k^T s_k}, \\ &= B_k s_k - \frac{(B_k s_k)(s_k^T B_k^T s_k)}{s_k^T B_k s_k} + \frac{y_k(y_k^T s_k)}{y_k^T s_k}, \\ &= B_k s_k - B_k s_k + y_k, \\ &= y_k. \end{aligned}$$

□

Exercise 2.1: Consider the problem,

$$\begin{aligned} \text{minimize: } f(x) &= x_1^2 + x_1^2 x_3^2 + 2x_1 x_2 + x_2^4 + 8x_2 \\ \text{subject to: } 2x_1 + 5x_2 + x_3 &= 3. \end{aligned}$$

- (i) Determine which of the following points are stationary points: $(0, 0, 2)^T$, $(0, 0, 3)^T$, and $(1, 0, 1)^T$.

Solution. Computing the gradient and Hessian,

$$\begin{aligned} \nabla f(x) &= \begin{pmatrix} 2x_1 + 2x_1 x_3^2 + 2x_2 \\ 2x_1 + 4x_2^3 + 8 \\ 2x_1^2 x_3 \end{pmatrix} \\ \nabla^2 f(x) &= \begin{pmatrix} 2x_3^2 + 2 & 2 & 4x_1 x_3 \\ 2 & 12x_2^2 & 0 \\ 4x_1 x_3 & 0 & 2x_1^2 \end{pmatrix} \end{aligned}$$

Using the Lagrange Multipliers Necessary condition for optimality we need to see if there exists a scalar λ such that $\nabla f(x_*) = A^T \lambda$. Note that for our problem $A^T = (2, 5, 1)^T$. Checking $(0, 0, 2)^T$ we get,

$$\nabla f(x) = \begin{pmatrix} 2(0) + 2(0)(2)^2 + 2(0) \\ 2(0) + 4(0)^3 + 8 \\ 2(0)^2(0) \end{pmatrix} = \begin{pmatrix} 0 \\ 8 \\ 0 \end{pmatrix} \neq \begin{pmatrix} 2 \\ 5 \\ 1 \end{pmatrix} \lambda.$$

Clearly there is no value for λ which will make this expression true. So $(0, 0, 2)^T$ is not a stationary point.

Checking $(0, 0, 3)^T$ again we get,

$$\nabla f(x) = \begin{pmatrix} 2(0) + 2(0)(2)^2 + 2(0) \\ 2(0) + 4(0)^3 + 8 \\ 2(0)^2(0) \end{pmatrix} = \begin{pmatrix} 0 \\ 8 \\ 0 \end{pmatrix} \neq \begin{pmatrix} 2 \\ 5 \\ 1 \end{pmatrix} \lambda.$$

Again we have the same situation where no value for λ which will satisfy this expression so $(0, 0, 2)^T$ is not a stationary point.

Checking $(1, 0, 1)^T$ we get,

$$\nabla f(x) = \begin{pmatrix} 2(1) + 2(1)(1)^2 + 2(0) \\ 2(1) + 4(0)^3 + 8 \\ 2(1)^2(1) \end{pmatrix} = \begin{pmatrix} 4 \\ 10 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 5 \\ 1 \end{pmatrix} (2).$$

As we see when we set $\lambda = 2$ we get that expression is satisfied so $(1, 0, 1)^T$ is indeed a minimizer.

□

- (ii) Determine whether each stationary point is a local minimizer, local maximizer, or saddle point.

Proof. To see what type of stationary point $x_* = (1, 0, 1)^T$ is we need to observe $Z^T \nabla^2 f(x_*) Z$ where Z is the null space matrix for A . Note that Z can look like the following,

$$Z = \begin{pmatrix} -\frac{1}{2} & -\frac{5}{2} \\ 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Computing $Z^T \nabla^2 f(x_*) Z$ we get,

$$\begin{aligned} Z^T \nabla^2 f(x_*) Z &= \begin{pmatrix} -\frac{1}{2} & 0 & 1 \\ -\frac{5}{2} & 1 & 0 \end{pmatrix} \begin{pmatrix} 2(1)^2 + 2 & 2 & 4(1)(1) \\ 2 & 12(0)^2 & 0 \\ 4(1)(1) & 0 & 2(1)^2 \end{pmatrix} \begin{pmatrix} -\frac{1}{2} & -\frac{5}{2} \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \\ &= \begin{pmatrix} -\frac{1}{2} & 0 & 1 \\ -\frac{5}{2} & 1 & 0 \end{pmatrix} \begin{pmatrix} 4 & 2 & 4 \\ 2 & 0 & 0 \\ 4 & 0 & 2 \end{pmatrix} \begin{pmatrix} -\frac{1}{2} & -\frac{5}{2} \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \\ &= \begin{pmatrix} -1 & -6 \\ -6 & 15 \end{pmatrix} \end{aligned}$$

Quickly solving for eigenvalues we get the following characteristic equation, $(-1 - \lambda)(15 - \lambda) - 36 = \lambda^2 - 14\lambda - 51 = (\lambda + 3)(\lambda - 17)$ which gives eigenvalues -3 and 17 so the matrix is indefinite and x_* is a saddle point. □

Exercise P18: Apply the forward difference formula $f'(x) \approx (f(x+h) - f(x))/h$ to estimate the gradient of the function,

$$f(x) = \exp(10x_1 + 2x_2^2)$$

at point $x = (-1, 1)$. Assuming that $\epsilon_{mach} = 2.2204 \times 10^{-16}$ on the computer you used, how accurate is the approximated gradient when you actually compute using the best value of h and the 'simpler' value of h , from page 425?

Solution. Applying the feed forward difference formula we get an approximate gradient of,

$$\nabla \hat{f}(x) = \begin{bmatrix} \frac{e^{10(x_1+h)+2x_2^2} - e^{10x_1+2x_2^2}}{h} \\ \frac{e^{10x_1+2(x_2+h)^2} - e^{10x_1+2x_2^2}}{h} \end{bmatrix}$$

Computing our estimated gradient with $hs = \sqrt{\epsilon_{mach}}$ (simple h) value we get the following,

Console:

```
% Defining original inline function
>> f = @(x, y) exp(10*x + 2*y^2)
% Setting 'simple' h value
>> h = sqrt(eps)
% Defining simple feed forward difference formula
>> fsimple = @(x, y) [(f((x + h), y) - f(x, y))/h,
                      (f(x, (y + h)) - f(x, y))/h];

% Computing 'simple' estimation of gradient
>> dfs = fsimple(-1,1)
dfs =
    0.003354626529472    0.001341850551398
```

The 'complex' value for h is given by the following,

$$h_1 = \sqrt{\frac{2|f(x)|\epsilon}{|\frac{d^2f}{dx_1^2}(x)|}},$$

$$h_2 = \sqrt{\frac{2|f(x)|\epsilon}{|\frac{d^2f}{dx_2^2}(x)|}}.$$

The partial derivatives that we will use are,

$$\frac{d^2f}{dx_1^2}(x) = 100e^{10x_1+2x_2^2},$$

$$\frac{d^2f}{dx_2^2}(x) = e^{10x_1+2x_2^2}(16x_2^2 + 4).$$

Computing the separate h values and plugging into our feed forward difference formula we get the following,

Console:

```
% Defining original inline function
>> f = @(x, y) exp(10*x + 2*y^2)

% Defining second order partial derivatives
>> dfdx = @(x, y) 100*exp(10*x + 2*y^2);
>> dfdy = @(x, y) exp(10*x + 2*y^2)*(16*y^2 + 4);

% Computing 'complex' h value
>> h1 = sqrt((2*abs(f(-1,1))*eps)/abs(dfdx(-1,1)))
h1 =
    2.107342425544702e-09

>> h2 = sqrt((2*abs(f(-1,1))*eps)/abs(dfdy(-1,1)))
h2 =
    4.712160915387242e-09

% Defining 'complex' feed forward difference formula
>> fcomplex = @(x, y) [(f((x + h1), y) - f(x, y))/h1,
                        (f(x, (y + h2)) - f(x, y))/h2];

$ Computing 'complex' estimation of gradient
>> dfc = fcomplex(-1,1)
dfc =
    0.003354626263876    0.001341850507582
```

Comparing our two approximation of the gradient we find that the more 'complex' h value led to a single order of magnitude smaller error than the 'simple' h value.

Console:

```
>> dfforrealtime = @(x, y) [10*exp(10*x + 2*y^2),
                             4*y*exp(10*x + 2*y^2)]
dfforrealtime =
function_handle with value:

    @(x,y)[10*exp(10*x+2*y^2),4*y*exp(10*x+2*y^2)]

>> df = dfforrealtime(-1,1)
df =
    0.003354626279025    0.001341850511610

>> norm(df - dfc)
ans =
```

1.567514002442148e-11

```
>> norm(df - dfs)
ans =
    2.535874569394526e-10
```

□

Exercise P19: Suppose a user want to solve $\min_{x \in \mathbb{R}^n}$ for a smooth objective function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, but they do not provide a gradient function $\nabla f(x)$. We can still use quasi-Newton if we apply finite differences to replace the missing gradient. Use this idea to write a new code,

```
function [xk, xklist] = sr1btfd(x0, f, tol, maxiters)
```

which replaces the user-provided gradient in `sr1bt.m` with a finite-difference gradient. Test it on the Rosenbrock example, as is done `rosencompare.m`, and give iterations. (Don't worry about visualizing, or the contours.) How would it help if the user provided values for the typical scales of the gradient and the Hessian.

Solution. The following is a modification of the `sr1bt.m` code removing instances where the gradient is computed with the user supplied function, now we estimate the gradient with a finite difference gradient using a simple value for $h = \sqrt{\epsilon_{mach}}$.

Code:

```
function [xk, xklist] = sr1btfd(x0, f, tol, maxiters)
% This code is a modification of SR1BT
% only requires the user to provide an inline function f,
% initial value x0, gradient tolerance tol, and maximum
% iterations maxiters. For computing the gradient it uses
% the finite difference formula.

if nargin < 4
    maxiters = 20000; % never take more steps than this
end
xk = x0(:); % force into column shape
if nargin > 1
    xklist = [xk];
end
n = length(xk);
h = sqrt(eps);

B = eye(n, n); % first step is steepest-descent: B=I
fxk = f(xk);
```

```

dfxk = finitedifference(xk, f, h);

for k = 1:maxiters
    if norm(dfxk) < tol          % absolute tolerance on gradient f
        break
    end
    pk = - B \ dfxk;            % quasi-Newton step
    if dfxk' * pk >= 0.0
        fprintf('warning: non-descent direction at step %d (use steepest-descent)
        pk = - dfxk;
    end
    alpha = bt(xk,pk,dfxk,f);    % back-tracking line search
    oldxk = xk;
    olddfxk = dfxk;
    xk = xk + alpha * pk;        % do update
    if nargout > 1
        xklist = [xklist xk];  % append latest point to list
    end
    fxk = f(xk);
    dfxk = finitedifference(xk, f, h);
    sk = xk - oldxk;            % sk, yk used to update B; see GNS p. 413
    yk = dfxk - olddfxk;
    v = yk - B * sk;
    denom = v' * sk;
    if denom == 0
        break                  % in this case we are certainly done
    end
    if abs(denom) < 1.0e-14
        warning(strcat('divide by near-zero in symmetric rank-one update ',...
            ' at iteration %d; convergence?'),k)
    end
    B = B + v * (v / denom)';    % symmetric rank-one update
end
end % function

function alpha = bt(xk,pk,dfxk,f)
% BT Apply backtracking using standard default parameters.
Dk = dfxk' * pk;
fxk = f(xk);
mu = 1.0e-4; % modest sufficient decrease
rho = 0.5;   % backtracking by halving
alpha = 1.0; % alpha_0=1 because of Newton, and by Thm 11.7
while f(xk + alpha * pk) > fxk + mu * alpha * Dk
    alpha = rho * alpha;
end
end % function

```

```
% Finite Difference Function
function df = finitedifference(xk,f,h)
    n = length(xk);
    df = zeros(n,1);
    for i = 1:n
        xkh = xk;
        xkh(i) = xkh(i) + h;
        df(i) = (f(xkh) - f(xk))/h;
    end
end
```

Testing this code on the Rosenbrock example we get 26 iterations, just like if we had provided the gradient information as in SR1BT. (They actually do also provide different iterations, I checked by subtracting the xklist objects and seeing if they are nonzero)

Code:

```
>> rosencompare
SDBT: 6295 iterations
warning: non-descent direction at step 5 (use steepest-descent)
warning: non-descent direction at step 20 (use steepest-descent)
SR1BT: 26 iterations
warning: non-descent direction at step 5 (use steepest-descent)
warning: non-descent direction at step 20 (use steepest-descent)
SR1BTFD: 26 iterations
```

0	0
0.249999998137355	-0.000000186264515
0.415970409456413	0.101498858626165
0.394862837503768	0.213949383025268
0.434964939657060	0.187800019488852
0.448831332494431	0.192157743822965
0.571042488430562	0.304526663087369
0.588006589858421	0.333817150159347
0.712916763580310	0.478439632393395
0.672629877323743	0.447232824038971
0.676262138797526	0.452029931695006
0.830867419716175	0.664142416881233
0.749605286096544	0.560372318458381
0.778341708490561	0.602236537102965
0.836379267621450	0.690178819005310
0.852944462626156	0.719103554387714
0.888614185297360	0.793775093981369
0.891661920659544	0.798404604621669
0.954375360111830	0.906891377887991
0.955817951650525	0.911827242606205
0.954675744181637	0.912515018637703
0.989435149168922	0.977780880829016

0.989902743129441	0.979001186896564
1.005277285801859	1.010203001421890
1.000346050386857	1.000656199918521
1.000015560364472	1.000027730334830
0.999994746182567	0.999989379397641

If the user were to provide information about the scales for the gradient and the Hessian, we could see if $\sqrt{\epsilon_{mach}}$ is a good approximation in the finite differences formula for h . Recall that in the finite differences formula we have a more complex h value that is given by,

$$h_i = \sqrt{\frac{2|f(x)|\epsilon_{mach}}{\frac{d^2 f}{dx_i^2}(x)}}.$$

The scale of the Hessian, or its condition number would tell us how much these values for h vary in the finite difference formula. A condition number close to 1 suggests that all second order partial derivatives are of the same order of magnitude and therefore h_i are relatively similar in terms of size, in which case we might use the 'simple' formula for h . More directly if the value of the function and the value of the diagonal elements of the Hessian are of the same scale then the 'simple' h will be suitable. $\square$